École Supérieure en Informatique
ESI
Sidi Bel Abbès
المدرسة العليا للإعلام الآلي
سيدي بلعباس

NetTwin

A Digital Twin Architecture for
Intelligent, Fault-Tolerant IoT Networks

Supervised by:

Dr. Serhane Ousama

Jury Members:

Pr. [abdelatif rahmoun]

Dr. [Hamdan Bensenane]

Team Members:

Ghrib Khouloud Abdennbi Lamia Ammi Maissara Khiter Sara

Chapter 3

Edge Surveillance Node and Video Streaming Subsystem

3.1 Overview and Architectural Role

The video streaming and environment surveillance subsystem extends NetTwin with a physical-layer security dimension: continuous, AI-aware visual monitoring of the deployment environment. It operates as an intelligent edge node on a dedicated Raspberry Pi 3 Model B equipped with an official Raspberry Pi Camera Module v2.

Physically, the edge node joins the network via the Wi-Fi access point hosted by the NetTwin-Lab infrastructure (SSID: `NetTwin-Lab`). Data frames route through the Open vSwitch layer under the supervision of the Ryu SDN Controller. To ensure deterministic behaviour across the SDN flow tables and WebSocket endpoints, the camera node is assigned a fixed static IP address of `192.168.1.30` within the IoT subnet.

Integration with the Digital Twin

The camera node is not a standalone CCTV system — it is a first-class participant in the NetTwin digital twin. Motion events are reported to the shared state layer; the SDN Proxy enforces VLAN policy on all camera API calls; and the AI Engine can flag the node if its behaviour deviates from baseline. Visual telemetry is thus fully integrated into the same observability, security, and health prediction pipeline that governs every other IoT device.

3.2 Software Architecture

The local application (`stream.py`) is a multi-threaded Python 3 process. Its core dependencies are:

- **picamera2:** Native hardware interface for the Raspberry Pi Camera Module. Handles image sensor initialisation, calibration, and frame-buffer allocation.
- **OpenCV (cv2):** Computer vision engine for colour-space conversions (RGBA/BGR/Grayscale), Gaussian filtering, frame differencing, binary thresholding, and

JPEG encoding.

- **Flask:** Lightweight WSGI web server exposing the REST API and MJPEG streaming endpoint to the Admin PC dashboard.
- **threading:** Parallel execution pools isolating continuous streaming, DVR clip recording, and motion analysis into independent tracks.
- **requests:** Synchronous HTTP POST for telemetry payloads to the central server.
- **subprocess / ffmpeg:** Out-of-process transcoding of raw AVI captures to H.264 MP4 for browser-compatible playback.

3.3 Real-Time Video Streaming

3.3.1 On-Demand MJPEG Endpoint (`/video`)

The `/video` route hosts an HTTP MJPEG stream. On each request, a generator loop:

1. **Captures** an uncompressed frame via `picam2.capture_array()`.
2. **Compresses** it to 320 × 240 pixels at 50% JPEG quality using `cv2.imencode`.
3. **Encapsulates** the binary frame in a `multipart/x-mixed-replace` boundary block, allowing the browser to update the image in-place without WebSocket overhead.

3.3.2 Continuous Push-Frame Thread (`push_frames`)

A background daemon thread runs independently of the web endpoint, executing once per second:

1. Captures and compresses the current frame identically to the MJPEG path.
2. Issues a `requests.post` to the Admin PC receiver at `http://192.168.1.100:5000/api/camera/1` carrying headers `Content-Type: image/jpeg` and `X-IP: 192.168.1.30`.

This push path acts as a visual telemetry heartbeat: the central digital twin dashboard remains synchronised even when no user is actively watching the live stream.

3.4 Computer Vision Motion Detection Pipeline

3.4.1 Image Processing Algorithm

The motion detection engine runs every 0.5 seconds in a dedicated background thread (`motion_loop`). Each cycle applies a five-stage processing pipeline:

Frame Capture (RGBA) ⟶ BGR Conversion ⟶ Grayscale ⟶ Gaussian Blur ⟶ Frame Differen

1. **Grayscale conversion:** Reduces the frame to a single-channel 8-bit matrix, eliminating false triggers from lighting colour shifts.
2. **Gaussian smoothing:** A 21×21 kernel (`cv2.GaussianBlur`) suppresses high-frequency sensor noise, shadow flickers, and pixel artefacts.
3. **Absolute differencing:** Computes the pixel-level delta between the current blurred frame F_t and the previous baseline F_{t-1}:

$$\text{Diff} = |F_t - F_{t-1}|$$

4. **Binary thresholding:** Pixels with delta < 25 map to 0 (black); pixels with delta ≥ 25 map to 255 (white) via `cv2.threshold`.
5. **Density evaluation:** The change ratio is computed as:

$$\text{ratio} = \frac{\texttt{countNonZero}(\text{thresh})}{320 \times 240}$$

   A ratio exceeding **2%** constitutes a positive motion trigger.

3.4.2 Two-State Synchronisation Machine

To prevent alert flooding during a continuous intrusion event, the engine enforces a strict state machine protected by a concurrency lock (`motion_lock`):

Table 3.1: Motion detection state machine transitions

NavyBlue

CLEAR → DETECTED	Ratio > 2% while inactive	Sets `active = True`; fires `send_motion_alert(True)` in daemon thread; opens session entry in local JSON log
LightGray DETECTED → CLEAR	$\Delta t \geq 30$ s of unbroken stillness	Sets `active = False`; fires `send_motion_alert(False)`; closes and timestamps the session entry

The 30-second cooldown window prevents a single physical intrusion from generating dozens of fragmented alerts. The alert payload posted to `http://192.168.1.100:5000/api/iot/stat` carries:

Listing 3.1: Motion alert payload (JSON)

```
{
  "ip": "192.168.1.30",
  "motion": true,
  "motion_status": "DETECTED",    // or "CLEAR"
  "timestamp": "YYYY-MM-DD HH:MM:SS"
}
```

3.5 Local Session Logging

To ensure audit continuity during network or controller outages, the node maintains a file-backed session log at `/home/pi/motion_sessions.json`:

- **Session open (`open_session`):** On transition to DETECTED, appends {`"detected": ts, "cleared": None, "duration": None`} to the JSON array on disk.
- **Session close (`close_session`):** On CLEAR transition, performs a reverse scan for the open entry, computes elapsed duration (e.g. `"2m 14s"`), and writes the completed record back to disk.

Three REST endpoints expose this log to the Admin PC:

Table 3.2: Motion log REST endpoints

NavyBlue		
`/motion`	GET	Returns current binary state: `DETECTED` or `CLEAR`
LightGray `/motionlog`	GET	Returns total session count, ongoing sessions, and the 100 most recent records in reverse-chronological order
`/motionlog`	DELETE	Resets the log file to an empty array

3.6 Decentralised DVR and Automated Storage Management

3.6.1 Multi-Threaded Clip Recording and H.264 Transcoding

A parallel thread (`record_clips`) continuously archives video in 5-minute (300-second) segments:

1. **Raw capture:** Frames are recorded at 10 FPS using the XVID codec into a temporary `.avi` file named with a UTC timestamp prefix (e.g. `YYYY-MM-DD_HH-MM-SS_raw.avi`).
2. **H.264 transcoding:** Once the 300-second clip closes, `subprocess.run` invokes `ffmpeg` with optimisation flags (`-vcodec libx264 -preset ultrafast -pix_fmt yuv420p`) to produce a browser-ready `.mp4`.
3. **Housekeeping:** The intermediate `.avi` is deleted after the `.mp4` is confirmed, recovering disk space immediately.

3.6.2 Self-Healing Disk Watchdog

Before allocating each new clip, the thread queries available storage via `os.statvfs()`. If free space falls below **5.0 GB** (`MIN_FREE_GB = 5`), a FIFO cleanup loop deletes the oldest `.mp4` files in `/home/pi/recordings` until the threshold is satisfied. Because clip files are named by timestamp, alphabetical order maps exactly to chronological order, making the FIFO selection trivially correct.

With 5-minute clips at 10 FPS and H.264 encoding, typical storage consumption is 150–300 MB per clip depending on scene complexity. The automated watchdog prevents file-system exhaustion, but operators should size the partition to accommodate the desired retention window before deploying in production.

3.7 Full REST API Reference

Table 3.3: Camera edge node — complete REST API

NavyBlue		
`/video`	GET	Live MJPEG stream at 320×240, 50% JPEG quality
LightGray `/motion`	GET	Instant motion state (`DETECTED` / `CLEAR`)
`/motionlog`	GET	Session history: totals, ongoing count, last 100 entries
LightGray `/motionlog`	DELETE	Clears all motion session records
`/recordings`	GET	Lists all `.mp4` clips with count and free storage (GB)
LightGray `/recordings/<filename>`	GET	Streams a specific clip for in-browser HTML5 playback
`/recordings/<filename>`	DELETE	Removes a specific clip from disk
LightGray `/recordings`	DELETE	Purges all clips from the recordings directory